

Decorators in Python

From First-Class Functions to CPython, Production Design, and Interview
Mastery

Mahesh Kumar. P

Contents

1	Decorators in Python	1
1.1	Motivation: The Cross-Cutting Code Problem	1
1.1.1	Problem, pain, and need	1
1.2	Required Prerequisites: Names, Objects, Calls, and Scope	2
1.2.1	Names point to objects	2
1.2.2	First-class and higher-order functions	2
1.2.3	Callable objects and <code>__call__</code>	3
1.2.4	Nested functions, LEGB, and closures	3
1.2.5	Execution stack and frames	3
1.3	The Birth of Decorators	4
1.3.1	Evolution, not magic	4
1.4	Decorator Intuition: Useful Analogies with Boundaries	4
1.5	Internal Working of the At-Sign	5
1.6	CPython Internals Without Mythology	6
1.6.1	Compile time and module execution	6
1.6.2	Function and code objects	6
1.6.3	Frames, calls, reference counts, and collection	6
1.7	Decorator Syntax Through Complete Worked Examples	7
1.7.1	Worked Example I: A transparent timing decorator	7
1.7.2	Why <code>functools.wraps</code> matters	8
1.7.3	Worked Example II: Parameterized retry	9
1.7.4	Decorating methods and descriptors	10
1.7.5	Class decorators and decorating callable classes	11
1.8	Closures Inside Decorators	11
1.9	Stacking Decorators	12
1.10	Class-Based Decorators	13
1.11	Advanced Production Patterns	13
1.11.1	Memoization from scratch, then the standard tool	14
1.11.2	Type-preserving decorator signature	14
1.12	Performance Considerations	15
1.13	Best Practices, Testing, and Debugging	15
1.14	Common Misconceptions and Forty-Five Failure Modes	16

1.15 Interview Preparation	18
1.16 Practice Problems and Mini-Projects	19
1.16.1 Easy	19
1.16.2 Medium	19
1.16.3 Hard and interview-grade	19
1.16.4 Debugging and output prediction	19
1.16.5 Capstone mini-projects	20
1.17 Industry Applications	20
1.18 When Decorators Are and Are Not the Right Tool	21
1.19 Complete Cheat Sheet and Revision System	21
1.19.1 Quick reference table	21
1.19.2 Mind map	22
1.19.3 Decision tree	22
1.19.4 CPython internal flow summary	22
1.19.5 Revision, design, and interview checklists	22

Chapter 1

Decorators in Python

Chapter Promise

This chapter does not begin with the at-sign. It begins with repeated, fragile code and lets the decorator idea evolve naturally. By the end, you will be able to derive decorators from first principles, trace their memory and execution behavior, preserve metadata, design parameterized and class-based decorators, explain CPython’s role accurately, and decide when a decorator is the wrong abstraction.

1.1 Motivation: The Cross-Cutting Code Problem

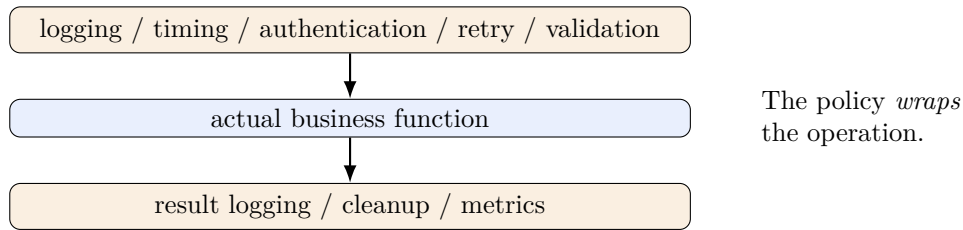
Imagine a research platform with functions for loading data, fitting a model, exporting a report, and deleting an experiment. Production requirements arrive one by one: log every call, time slow functions, authenticate users, check permissions, validate arguments, retry transient failures, limit request rates, profile hot paths, and cache expensive results.

1.1.1 Problem, pain, and need

Without a reusable mechanism, each function acquires the same scaffolding:

```
1 def export_report(report_id, user):
2     if not user.is_authenticated:
3         raise PermissionError("Sign in first")
4     if "export" not in user.permissions:
5         raise PermissionError("Missing export permission")
6     start = perf_counter()
7     logger.info("export_report started")
8     try:
9         result = really_export(report_id)
10        logger.info("export_report succeeded")
11        return result
12    finally:
13        logger.info("elapsed=%f", perf_counter() - start)
```

The business operation is one line; the policy code surrounds it. Copy-and-paste creates inconsistent fixes, forgotten checks, noisy tests, and functions that violate single responsibility. A helper can reduce duplication, but callers must remember to invoke it in the right order. Inheritance is a poor fit: free functions are not naturally related by an “is-a” hierarchy, Python permits only one concrete method-resolution order, and authentication, timing, retry, and caching are independent policies.



The idea before its name

We want to take an existing callable, place reusable behavior before and/or after its call, and obtain another callable with the enhanced behavior—without editing the original body. That transformation is the seed from which decorators grow.

1.2 Required Prerequisites: Names, Objects, Calls, and Scope

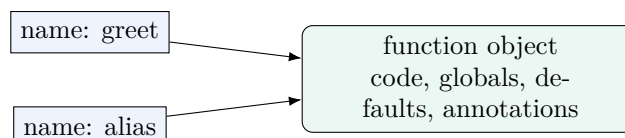
1.2.1 Names point to objects

Python variables are names bound to objects, not boxes that permanently contain values. A function definition creates a function object and binds a name to it.

```

1 def greet(name):
2     """Return a greeting."""
3     return f"Hello, {name}"
4
5 alias = greet
6 print(alias is greet) # True
7 print(callable(alias)) # True
8 del greet
9 print(alias("Ada")) # still works

```



`alias is greet` compares identity: both names reference the same object. `id()` exposes an implementation-oriented identity number useful for experiments, not business logic. Deleting `greet` removes one binding; it does not destroy the object while `alias` still references it.

1.2.2 First-class and higher-order functions

A value is first-class when it can be stored, passed, returned, and placed in collections. Functions satisfy all four:

```

1 def shout(text):
2     return text.upper()
3
4 operation = shout # store
5 pipeline = [shout, str.strip] # collect
6
7 def apply(operation, value): # pass as argument
8     return operation(value)
9

```

```

10 def choose(loud): # return a function
11     return shout if loud else str.lower

```

A function that accepts or returns a function is a *higher-order function*. Decorators are a disciplined application of this capability, not a separate magic feature.

1.2.3 Callable objects and `__call__`

Functions are callable, but functions are not the only callable objects. If an instance's class defines `__call__`, the instance may be invoked with parentheses.

```

1 class Multiplier:
2     def __init__(self, factor):
3         self.factor = factor
4
5     def __call__(self, value):
6         return self.factor * value
7
8 triple = Multiplier(3)
9 print(callable(triple), triple(7)) # True 21

```

Conceptually, `triple(7)` dispatches through the type's call protocol. Do not manually rewrite every call as `triple.__call__(7)`: special-method lookup has type-level rules, and normal call syntax is the protocol.

1.2.4 Nested functions, LEGB, and closures

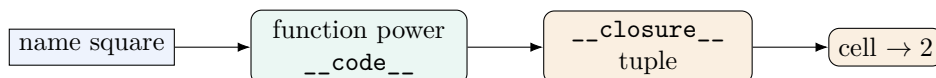
Name lookup follows the relevant part of LEGB: Local, Enclosing, Global, Builtins. A nested function can use a name from its lexically enclosing function.

```

1 def make_power(exponent):
2     def power(base):
3         return base ** exponent
4     return power
5
6 square = make_power(2)
7 cube = make_power(3)
8 print(square(5), cube(2)) # 25 8

```

The returned `power` outlives the call that created it. Its captured `exponent` is stored through a closure cell, not kept alive by a permanently suspended ordinary stack frame.



1.2.5 Execution stack and frames

Each active Python call has a frame containing execution state, local variables, a link to code, and bookkeeping. A simplified call sequence is:

```

module frame
-> make_power frame (local exponent=2)
    -> creates power with a cell for exponent
<- frame returns and becomes collectible
-> power frame (local base=5; free exponent resolved via cell)

```

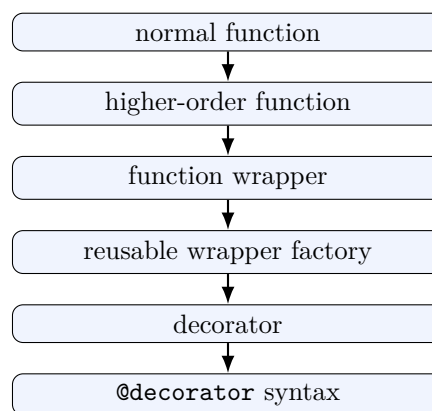
Late binding

Closures capture variables (cells), not frozen textual values. Loop-created closures often all observe the final loop value. Bind a snapshot with a default argument, for example `lambda x, i=i: x + i`, when snapshot semantics are intended.

1.3 The Birth of Decorators

1.3.1 Evolution, not magic

Start with a normal function. Then pass it to a higher-order function. Place the call inside a nested wrapper. Return that wrapper. Rebind the original public name. Finally, use syntax that expresses that rebinding clearly.



```

1 def announce(function):
2     def wrapper():
3         print("before")
4         result = function()
5         print("after")
6         return result
7     return wrapper
8
9 def hello():
10    print("hello")
11
12 hello = announce(hello)
13 hello()
  
```

Higher-order functions were technically sufficient: the last assignment already works. The remaining pain was human and structural—repetition, easy-to-miss rebinding, distance between a definition and its policies, and awkward stacking. Decorator syntax standardizes and localizes that transformation.

1.4 Decorator Intuition: Useful Analogies with Boundaries

Analogy	What it explains	Where it breaks
Gift wrapping	Original value is enclosed; layers can stack.	Wrapping software may change call semantics and identity.

Phone case	Adds protection without rebuilding the phone.	A wrapper is often a new callable, not merely a passive shell.
Security guard	Checks access before entry.	Many decorators also transform results or handle exceptions afterward.
Airport security	Ordered policies surround a journey.	Decoration usually happens at definition time; checks happen at call time.
Waiter	Intermediary receives a request and returns a result.	The wrapped function may still be reachable through <code>__wrapped__</code> .
Middleware	Request/response behavior nests around an endpoint.	Framework middleware may use separate protocols and lifecycle hooks.
Proxy	Controls and forwards access to a target.	A closure wrapper may preserve less interface behavior than a full proxy.
Packaging	Public presentation changes while core remains.	Decoration can deliberately replace the core with any object.
Assembly line	Each stage transforms the product in order.	Function decorators form nested calls, not always a flat pipeline.

Working definition earned from the problem

A decorator is a callable used to transform a function or class definition. It receives the newly created object (possibly after configuration), and the name defined by the statement is bound to the decorator's returned object.

1.5 Internal Working of the At-Sign

Consider:

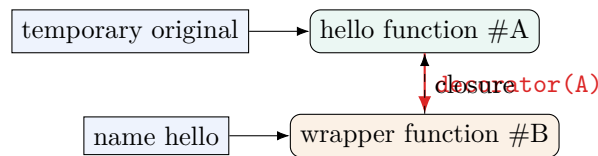
```
1 @decorator
2 def hello():
3     pass
```

Its essential meaning is:

```
1 def hello():
2     pass
3
4 hello = decorator(hello)
```

The tiny steps are important:

1. During execution of the containing scope, Python evaluates the decorator expression and retains its result.
2. It creates the function object from the compiled code object, current globals, defaults, annotations, and closure.
3. It calls the retained decorator callable with that function object.
4. It binds `hello` in the current namespace to the returned object.
5. If the decorator returned a wrapper, normal lookup of `hello` now finds the wrapper. The original survives while referenced by the wrapper closure or elsewhere.



References obtained before manual rebinding still point to the original. References obtained through the public name afterward point to the returned object. A decorator may return the same object, a wrapper, a callable instance, a class, or even a non-callable (legal but usually surprising).

1.6 CPython Internals Without Mythology

1.6.1 Compile time and module execution

CPython compiles source to bytecode/code objects before executing the module. Decoration itself occurs when the `def` statement executes, normally during module import. The exact bytecode opcodes vary by CPython release; the language-level invariant is evaluation of decorator expressions, function creation, decorator calls from the closest decorator outward, and final name binding.

1.6.2 Function and code objects

At the C implementation level, a Python function is represented by a `PyFunctionObject`. Conceptually it references:

- a `PyCodeObject` containing bytecode and static metadata;
- the globals dictionary used by global-name lookup;
- defaults, keyword defaults, annotations, module, name, and qualified name;
- an optional tuple of closure cell objects.

The code object's `co_cellvars` identifies locals that must be exposed to children; `co_freevars` identifies names obtained from enclosing cells. Inspect these properties rather than assuming a particular address or opcode sequence.

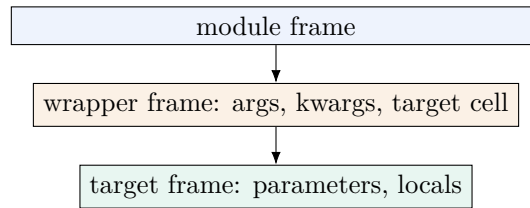
```

def outer(secret):
    def inner():
        return secret
    return inner

f = outer("token")
print(f.__code__.co_freevars) # ('secret',)
print(f.__closure__[0].cell_contents) # token
  
```

1.6.3 Frames, calls, reference counts, and collection

Calling a wrapper activates a wrapper frame; calling the captured target activates another frame. Thus one decorated invocation adds at least one Python-level call boundary per ordinary wrapper. CPython primarily reclaims objects through reference counting and supplements it with cyclic garbage collection. A closure reference raises the lifetime of the original function and its cells. Cyclic GC matters only if decorators create cycles; ordinary wrapper-to-target references are not inherently cyclic.



1.7 Decorator Syntax Through Complete Worked Examples

Short equivalence fragments establish syntax; the substantial examples below use the full learning protocol: problem, need, visualization, logic, pseudocode, implementation, driver, output, explanation, dry run, memory, flow, complexity, edge cases, mistakes, and interview discussion.

1.7.1 Worked Example I: A transparent timing decorator

Problem Statement. Several functions require timing, and copied timing code obscures their purpose.

Why needed. A reusable policy must accept arbitrary positional and keyword arguments, return the target's value, and preserve introspection metadata.

Visualization.

```

caller -> timed wrapper -> original function -> result
          | start | stop, report
  
```

Step-by-step logic. Capture the target in a closure; on each call record a start value, forward arguments, return the result, and report elapsed time in `finally` so exceptions do not suppress metrics.

Algorithm thinking. Decoration is a one-time transformation. Measurement is per invocation. The wrapper must not swallow the target exception.

Language-independent pseudocode.

```

DECORATE target:
  CREATE wrapper accepting all arguments
  start <- monotonic clock
  TRY: RETURN target(all arguments)
  FINALLY: print(clock - start)
  COPY metadata from target to wrapper
  RETURN wrapper
  
```

Complete Python implementation and driver code.

```

1 from functools import wraps
2 from time import perf_counter
3
4 def timed(function):
5     @wraps(function)
6     def wrapper(*args, **kwargs):
7         start = perf_counter()
8         try:
9             return function(*args, **kwargs)
10        finally:
11            elapsed = perf_counter() - start
12            print(f"{function.__name__}: measured")
13    return wrapper
14
  
```

```

15 @timed
16 def add(left: int, right: int = 0) -> int:
17     """Add two integers."""
18     return left + right
19
20 print(add(4, right=5))
21 print(add.__name__)

```

Expected console output.

```

add: measured
9
add

```

Output explanation. The first line is emitted by the wrapper's `finally`; the second is the forwarded return value; the third proves `wraps` preserved the public name.

Dry run table.

Step	State/action	Result
Definition	<code>timed(original_add)</code> creates wrapper	<code>add</code> -> wrapper
Call	wrapper receives <code>args=(4,)</code> and <code>kwargs={'right': 5}</code>	starts clock
Forward	original computes <code>4+5</code>	9
Finally	wrapper prints metric	exception status unchanged
Return	wrapper returns original result	9

Memory before execution. The module name `timed` points to the decorator; no wrapper exists for `add` before its `def` executes.

Memory after execution. `add` points to wrapper B, and B's closure points to original A. The standard unwrapping attribute installed by `wraps` also points from B to A.

Internal execution flow. `add(...)` resolves B, creates a B frame, B resolves function through a cell, creates A's frame, then unwinds A followed by B.

Complexity analysis. If the target costs $T(n)$ time and $S(n)$ auxiliary space, the decorated call remains $T(n) + O(1)$ time and $S(n) + O(1)$ active wrapper state. Real constant overhead can matter in tiny hot functions.

Edge cases. Exceptions, generators, coroutines, descriptors, and functions with unusual signatures need deliberate semantics. This timer measures generator creation, not later iteration; async functions need an `async def` wrapper and `await`.

Common mistakes. Forgetting `return`, accepting no arguments, using `time.time()` for elapsed durations, swallowing exceptions, or omitting `wraps`.

Interview discussion. State the contract first: `Callable -> Callable`; explain the closure; mention metadata and exception transparency; then discuss `async/generator` variants.

1.7.2 Why `functools.wraps` matters

Without metadata repair, introspection sees the generic wrapper:

```

1 def noisy(function):
2     def wrapper(*args, **kwargs):
3         return function(*args, **kwargs)
4     return wrapper

```

`functools.wraps(function)` is convenience syntax for `update_wrapper(wrapper, function)`. It normally updates `__name__`, `__qualname__`, `__doc__`, `__annotations__`, `__type_params__` on supporting Python versions, merges `__dict__`, and adds `__wrapped__`. `__module__` is also assigned. The exact default assigned set is version-dependent.

```

1 import inspect
2
3 print(add.__name__) # add
4 print(add.__doc__) # Add two integers.
5 print(add.__annotations__) # copied annotations
6 print(add.__module__) # defining module
7 print(add.__wrapped__) # original function
8 print(inspect.signature(add)) # follows __wrapped__ by default
9 print(inspect.unwrap(add)) # deepest wrapped object

```

Metadata is not behavior

`wraps` improves identity-like metadata and tool interoperability; it does not give a `*args`, `**kwargs` wrapper the original call semantics at the interpreter level. Static typing may require `ParamSpec`; exact runtime signatures may require specialized tooling or explicit `__signature__`.

1.7.3 Worked Example II: Parameterized retry

Problem Statement. Retry count and delay vary by operation. A plain decorator receives the function, leaving no place for configuration.

Why needed. We need a decorator factory: configuration now, target during definition, arguments during each call.

Visualization.

```

retry(attempts=3) -> decorate(function) -> wrapper(*args, **kwargs)
configuration target cell call data

```

Logic and algorithm. Validate configuration immediately. Catch only declared transient exceptions. Try at most `attempts` times, re-raising the final failure.

Pseudocode.

```

FACTORY attempts, exceptions:
    validate attempts
DECORATE target:
    CALL wrapper(arguments):
        FOR attempt 1..attempts:
            TRY RETURN target(arguments)
            CATCH configured error:
                IF final attempt: RAISE
        RETURN wrapper
    RETURN decorate

```

Implementation and driver.

```

1 from functools import wraps
2
3 def retry(*, attempts=3, exceptions=(OSError,)):
4     if attempts < 1:
5         raise ValueError("attempts must be at least 1")
6
7     def decorate(function):

```

```

8     @wraps(function)
9     def wrapper(*args, **kwargs):
10         for attempt in range(1, attempts + 1):
11             try:
12                 return function(*args, **kwargs)
13             except exceptions:
14                 print(f"failure {attempt}")
15                 if attempt == attempts:
16                     raise
17         return wrapper
18     return decorate
19
20 state = {"calls": 0}
21
22 @retry(attempts=3, exceptions=(ConnectionError,))
23 def fetch():
24     state["calls"] += 1
25     if state["calls"] < 3:
26         raise ConnectionError("temporary")
27     return "data"
28
29 print(fetch())

```

Expected output and explanation.

```

failure 1
failure 2
data

```

Calls one and two match the configured exception and retry; call three returns, so the wrapper stops immediately.

Dry run.

Attempt	target outcome	wrapper decision	calls
1	ConnectionError	print, continue	1
2	ConnectionError	print, continue	2
3	"data"	return	3

Memory before/after. Calling the factory creates cells for `attempts` and `exceptions`; decoration creates a cell for `function`; the returned wrapper reaches all three through its closure chain.

Internal flow. Import-time: factory $\rightarrow$ decorator $\rightarrow$ wrapper. Call-time: wrapper $\rightarrow$ target, possibly repeatedly.

Complexity. With at most k attempts, worst-case time is $O(kT(n))$ plus delay; extra decorator state is $O(1)$. External side effects may occur k times.

Edge cases. Never blindly retry non-idempotent writes, cancellation, programmer errors, or permanent authorization failures. Production retry usually needs exponential backoff, jitter, observability, a deadline, and an async-aware sleep.

Mistakes and interview discussion. The three-layer shape is the usual interview trap. Say aloud what each call receives. Distinguish `@retry` from `@retry()` and explain why broad `except Exception` can be dangerous.

1.7.4 Decorating methods and descriptors

An ordinary instance method is a function stored on a class. Attribute access binds the instance as `self`; a generic wrapper works if it forwards all arguments.

```
1 class Account:
2     @timed
3     def balance(self, currency="USD"):
4         return 100
```

Ordering matters with descriptor-producing built-ins:

```
1 class Temperature:
2     @staticmethod
3     @timed
4     def c_to_f(c):
5         return c * 9 / 5 + 32
6
7     @classmethod
8     @timed
9     def unit(cls):
10        return cls.__name__
11
12    @property
13    @timed
14    def kelvin(self):
15        return 273.15
```

This common ordering lets `timed` receive a function and then lets `staticmethod`, `classmethod`, or `property` create the descriptor. A decorator intentionally designed for descriptor objects can use a different order, but generic function decorators should not assume descriptors are plain functions.

1.7.5 Class decorators and decorating callable classes

A class statement also creates an object and binds a name, so class decorators follow the same transformation rule.

```
1 def register(registry):
2     def decorate(cls):
3         registry[cls.__name__] = cls
4         return cls
5     return decorate
6
7 plugins = {}
8
9 @register(plugins)
10 class CsvExporter:
11     pass
```

Returning the original class preserves identity and normal construction. A class decorator may instead return a replacement class or factory, but that can affect `isinstance`, subclassing, pickling, and tooling.

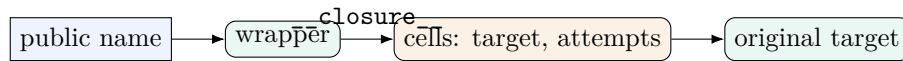
A callable class can itself be decorated because the decorator contract accepts callables, but a closure expecting `function.__name__` must tolerate instances without function metadata.

1.8 Closures Inside Decorators

Three different lifetimes are often present:

1. factory configuration survives through cells (for example, retry count);

- the original target survives through a wrapper cell;
- per-call locals live in the wrapper frame and disappear after that call unless captured elsewhere.



Use `nonlocal` to rebind captured state:

```

1 def count_calls(function):
2     calls = 0
3     @wraps(function)
4     def wrapper(*args, **kwargs):
5         nonlocal calls
6         calls += 1
7         wrapper.calls = calls
8         return function(*args, **kwargs)
9     wrapper.calls = 0
10    return wrapper
  
```

This state is shared by all callers of that decorated function and therefore requires synchronization if mutations must be thread-safe.

1.9 Stacking Decorators

```

1 @outer
2 @middle
3 @inner
4 def work():
5     pass
  
```

is equivalent to:

```

1 work = outer(middle(inner(work)))
  
```

Decorator expressions are evaluated in source order (top to bottom), while the produced callables are applied to the function from the nearest decorator upward (bottom to top). At call time, the outermost wrapper is entered first.

```

Definition/application: original -> inner -> middle -> outer -> bind work
Call entry: caller -> outer -> middle -> inner -> original
Return: caller <- outer <- middle <- inner <- original
  
```

Precise wording

Avoid the vague slogan “decorators execute bottom-up.” Decorator *expressions* are evaluated top-down; decorator *application* is bottom-up; wrapper call entry is outer-to-inner. These phases can print in different orders.

Order changes semantics. `@cache` outside `@authorize` can accidentally reuse a result without performing authorization on a cache hit. A transaction outside retry may keep all attempts in one transaction; retry outside transaction can create one transaction per attempt. Design the nesting intentionally.

1.10 Class-Based Decorators

Closure decorators store state in cells; decorator instances store it in attributes.

```

1 from functools import update_wrapper
2
3 class CountCalls:
4     def __init__(self, function):
5         self.function = function
6         self.calls = 0
7         update_wrapper(self, function)
8
9     def __call__(self, *args, **kwargs):
10        self.calls += 1
11        return self.function(*args, **kwargs)
12
13 @CountCalls
14 def ping():
15     return "pong"

```

	Closure decorator	Class-based decorator
State	cells/nonlocal variables	visible instance attributes
Readability	compact for small behavior	clear for substantial state/methods
Binding	functions naturally implement descriptors	instances need <code>__get__</code> to act like bound methods
Metadata	<code>@wraps</code>	<code>update_wrapper</code> or explicit properties
Testing	inspect closure or exposed API	state is straightforward to inspect
Cost	function plus cells	instance plus attribute dictionary/slots

The method-binding trap

If `@CountCalls` decorates a method, the class stores a callable instance, not a function descriptor. Implement `__get__` (often using `types.MethodType`) or return a function wrapper if automatic `self` binding is required.

1.11 Advanced Production Patterns

Pattern	Decorator responsibility	Professional concerns
Memoization	Map argument keys to results.	Hashability, memory growth, invalidation, concurrency.
LRU cache	Evict least-recently-used keys.	Use <code>functools.lru_cache</code> ; size and staleness policy.
Timing/profiling	Measure duration or collect samples.	Monotonic clock, async/generator semantics, low-noise metrics.
Logging	Record call context and outcome.	Secrets/PII, log volume, structured fields, exception fidelity.

Retry	Repeat transient failures.	Idempotency, backoff, jitter, deadlines, cancellation.
Singleton	Control instance creation.	Global state, testing, subclassing, serialization; often prefer dependency scope.
Authentication	Establish identity.	Request context, failure response, never trust client claims.
Authorization	Check permission on resource/action.	Check at call time, object-level policy, audit trail.
Validation	Reject malformed inputs.	Error quality, signature/type integration, duplicate validation.
Transactions	Begin/commit/rollback around work.	Nested transactions, isolation, async context, retries.
Dependency injection	Resolve and supply collaborators.	Hidden dependencies, lifetime/scoping, test overrides.
Rate limiting	Admit calls under a quota.	Atomic counters, distributed clocks, fairness, response metadata.
Registration	Add functions/classes to a registry at definition time.	Import order, duplicate keys, plugin discovery.
AOP	Apply cross-cutting concerns at join points.	Discoverability, ordering, debugging, excessive magic.

1.11.1 Memoization from scratch, then the standard tool

```

1 def memoize(function):
2     cache = {}
3     @wraps(function)
4     def wrapper(*args, **kwargs):
5         key = (args, tuple(sorted(kwargs.items())))
6         if key not in cache:
7             cache[key] = function(*args, **kwargs)
8         return cache[key]
9     wrapper.cache = cache
10    return wrapper

```

This educational version fails for unhashable arguments, can grow without bound, and may duplicate work under concurrency. Prefer:

```

1 from functools import lru_cache
2
3 @lru_cache(maxsize=256)
4 def parse_schema(text):
5     return expensive_parse(text)

```

Cache only functions whose output is stable for the full logical key. Time, random data, mutable globals, user identity, database changes, and side effects invalidate the simple mathematical model.

1.11.2 Type-preserving decorator signature

```

1 from collections.abc import Callable
2 from functools import wraps
3 from typing import ParamSpec, TypeVar
4

```

```
5 P = ParamSpec("P")
6 R = TypeVar("R")
7
8 def traced(function: Callable[P, R]) -> Callable[P, R]:
9     @wraps(function)
10     def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
11         print(function.__qualname__)
12         return function(*args, **kwargs)
13     return wrapper
```

Use **Concatenate** when the decorator inserts or removes a leading parameter. Static typing documents the transformation; runtime behavior still depends on the implementation.

1.12 Performance Considerations

Each Python wrapper adds argument packing/forwarding, a frame/call boundary, closure lookup, policy work, and return/exception propagation. Stacked wrappers multiply the constant overhead. Decoration itself usually happens once; parameterized factories and closures allocate a few objects once per decorated definition. **wraps** copies metadata once, so its call-time cost is normally negligible.

Benchmark the scenario, not a toy conclusion:

```
1 from timeit import repeat
2
3 samples = repeat("f(1)", setup="from __main__ import f", number=1_000_000)
4 print(min(samples))
```

Warm up imports and caches, compare equivalent behavior, repeat measurements, report distributions, and profile before optimizing. A wrapper around a database request is negligible; five wrappers around a tiny numeric primitive in an inner loop may dominate.

Reasonable micro-optimizations include moving validation to decoration time, binding frequently used collaborators in closure locals, avoiding expensive logging argument construction when disabled, and combining inseparable policies. Do not remove clarity without measurement. For generators and async functions, use protocol-preserving wrappers rather than misleadingly fast incorrect ones.

1.13 Best Practices, Testing, and Debugging

Production checklist

- Give the decorator one clear responsibility and a verb/adjective name that reveals policy.
- Use **functools.wraps**; document arguments, ordering, side effects, exceptions, and sync/async support.
- Forward arguments and return values faithfully; preserve cancellation and tracebacks.
- Validate decorator configuration at definition time and call data at call time.
- Use **ParamSpec** and **TypeVar** for type-preserving decorators.
- Expose deliberate observability or control hooks such as **cache_info**, **reset**, or **__wrapped__**.
- Avoid storing request-specific mutable state globally or on a shared wrapper without

synchronization.

- Test undecorated policy logic, wrapper behavior, metadata, exceptions, ordering, concurrency, and edge protocols.

Debug from the outside inward:

1. print `repr(callable)`, `__qualname__`, and `inspect.signature`;
2. walk the `__wrapped__` chain with `inspect.unwrap`;
3. inspect `__closure__` and cell contents cautiously;
4. add breakpoints at each wrapper entry and exit;
5. temporarily call `function.__wrapped__` in tests only when bypassing policy is explicitly safe;
6. minimize the decorator stack and restore layers one by one.

1.14 Common Misconceptions and Forty-Five Failure Modes

#	Mistake / why	Symptom / debug	Professional solution
1	Thinking <code>@d</code> runs on every call.	Side effect appears only at import; trace definition.	Separate decoration-time and call-time code.
2	Calling target in decorator body.	Function runs during import.	Call it inside the returned wrapper.
3	Returning target result from decorator.	Name becomes data/non-callable.	Return a callable wrapper.
4	Forgetting to return wrapper.	Decorated name is <code>None</code> .	End decorator with <code>return wrapper</code> .
5	Forgetting wrapper's return.	Caller receives <code>None</code> .	<code>return function(...)</code> .
6	Wrapper accepts no arguments.	<code>TypeError</code> on parameters.	Forward <code>*args</code> , <code>**kwargs</code> or preserve signature.
7	Dropping keyword arguments.	Keyword calls fail.	Forward both argument forms.
8	Reordering arguments accidentally.	Wrong values/binding errors.	Forward unchanged; test keyword-only parameters.
9	Omitting <code>wraps</code> .	Name/doc/signature show wrapper.	Apply <code>@wraps(function)</code> .
10	Believing <code>wraps</code> changes code signature.	Runtime errors remain.	Use correct forwarding; treat metadata separately.
11	Writing <code>@retry</code> for a factory.	Factory receives function as configuration.	Use <code>@retry(...)</code> or support both forms explicitly.
12	Adding an unnecessary factory layer.	Decorator object returned instead of wrapper.	Name what each layer receives.
13	Confusing expression evaluation/application.	Unexpected print order.	Trace definition and call phases separately.
14	Assuming top decorator enters last.	Logs show outer enters first.	Expand to nested calls.
15	Ignoring order-sensitive policy.	Auth/cache leak or wrong transaction scope.	Document and test stacking order.
16	Capturing a loop variable late.	All wrappers use final value.	Snapshot with default/factory scope.

17	Rebinding captured state without <code>nonlocal</code> .	<code>UnboundLocalError</code> .	Declare <code>nonlocal</code> or mutate a deliberate object.
18	Using mutable global state.	Tests interfere/races occur.	Use scoped state and synchronization.
19	Assuming closure retains whole frame.	Misleading memory model.	Think function $\rightarrow$ cells $\rightarrow$ values.
20	Expecting copied target.	Identity tests surprise.	Wrapper is usually a distinct object.
21	Using <code>id</code> as permanent identity.	Values differ across runs/reuse.	Use domain identifiers, not memory identity.
22	Decorating a method with a callable instance blindly.	Missing <code>self</code> .	Implement descriptor binding or use function wrapper.
23	Wrong <code>staticmethod</code> order.	Decorator receives descriptor unexpectedly.	Put generic function decorator below it.
24	Wrong <code>classmethod</code> order.	Object is not called as expected.	Choose order based on accepted input.
25	Wrong <code>property</code> order.	Property/decorator type error.	Usually put <code>@property</code> outermost.
26	Swallowing exceptions.	False success/ <code>None</code> .	Re-raise; catch only intended types.
27	Retrying every exception.	Bugs repeat and side effects duplicate.	Retry classified transient failures only.
28	Retrying non-idempotent writes.	Duplicate charges/messages.	Use idempotency keys or no automatic retry.
29	Timing generator creation.	Near-zero misleading time.	Wrap iteration or document measurement boundary.
30	Wrapping coroutine with sync wrapper.	Returns un-awaited coroutine/logs wrong span.	Use <code>async def</code> and <code>await</code> .
31	Blocking sleep in async retry.	Event loop freezes.	Use async sleep.
32	Logging secrets in arguments.	Credential/PII exposure.	Redact and allowlist structured fields.
33	Unbounded memoization.	Memory grows forever.	Set a bound/invalidation policy.
34	Caching mutable/impure results.	Stale or shared mutated data.	Cache only stable semantics; copy when required.
35	Building cache key from unsorted kwargs.	Equivalent calls miss cache.	Canonicalize binding with signature tools.
36	Assuming cache is process-wide.	Workers disagree.	Choose local/distributed storage deliberately.
37	Doing I/O at decoration time.	Imports become slow/fail.	Defer runtime work or make registration lightweight.
38	Depending on import order silently.	Registry is incomplete.	Make discovery explicit and detect duplicates.
39	Replacing a class unnecessarily.	<code>isinstance</code> /pickling surprises.	Mutate/register/return original where possible.
40	Using singleton decorator as DI.	Hidden global state hurts tests.	Prefer explicit lifetime-managed dependencies.
41	Decorating magic methods on an instance.	Operators ignore instance attributes.	Special lookup occurs on the type; decorate class definitions.
42	Assuming decorators enforce security universally.	Bypass via alternate entry point.	Enforce policy at trusted boundaries and defense-in-depth.
43	Testing only happy path.	Failures leak state/metadata.	Test exceptions, concurrency, cancellation, and order.

44	Too many invisible layers.	Tracebacks and ownership become opaque.	Keep policy small, documented, observable.
45	Using a decorator when explicit code is clearer.	Control flow becomes surprising.	Prefer context managers, composition, or direct calls when scope is local.

1.15 Interview Preparation

A strong 60-second whiteboard answer

Python functions are objects. A decorator is a callable transformation applied when a function or class definition executes. `@d` above `def f` is essentially `f = d(f)`. Most function decorators return a closure wrapper that captures the original function, performs behavior before or after it, forwards arguments and results, and uses `functools.wraps` for metadata. Stacked decorators become nested calls, so order affects semantics. The trade-off is reusable cross-cutting behavior versus extra indirection, call overhead, and hidden control flow.

Question / follow-up	Best-answer direction
Why were decorators invented?	To attach reusable policies to definitions declaratively; HOFs enabled it, syntax improved locality and composability.
What does <code>@d(x)</code> mean?	Evaluate <code>d(x)</code> to obtain a decorator, then apply it to the created function.
When does decoration happen?	When the containing definition statement executes, commonly module import; wrapper behavior happens on later calls.
Can a decorator return a non-function?	Yes, any object; continued callability depends on the returned object.
Closure versus class decorator?	Cells are concise; instances expose state/methods but method descriptor behavior needs care.
What does <code>wraps</code> do?	Copies/updates selected metadata and creates <code>__wrapped__</code> ; it does not magically preserve runtime signature behavior.
How do stacked decorators work?	Expand <code>@a @b def f</code> to <code>f=a(b(f))</code> ; distinguish expression evaluation, application, and call entry.
How do methods work?	Functions are descriptors; attribute access binds <code>self</code> . Generic wrappers preserve this if they remain functions.
How would you support <code>async</code> ?	Detect or define an <code>async</code> -specific decorator; await target and preserve cancellation. Avoid blocking calls.
Why can caching plus auth be unsafe?	If cache is outside auth, a hit may bypass inner authorization or mix principals unless identity is in the key.
Decorator vs context manager?	Decorator covers whole calls/definitions; context manager explicitly covers a lexical block and is clearer for partial scope/resources.
Decorator vs inheritance?	Decorator composes orthogonal behavior around callables/classes; inheritance models substitutable type relationships.
Can you unwrap?	If layers cooperate via <code>__wrapped__</code> , use <code>inspect.unwrap</code> ; bypassing policy may be unsafe.
What is the complexity?	Usually target complexity plus constant per-wrapper overhead; retry/cache can deliberately change asymptotics.

Whiteboard implementation challenge

Implement `once` so the first successful result is cached, exceptions are not cached, `None` is a valid result, metadata is preserved, and concurrent first calls do not execute the target twice. Follow-ups: make it per-instance, resettable, async-aware, and correctly typed.

1.16 Practice Problems and Mini-Projects

1.16.1 Easy

1. Write `announce` that prints entry and exit while preserving a result.
2. Predict definition-time and call-time output for two stacked printing decorators.
3. Repair a decorator that loses arguments, return value, name, and docstring.
4. Create `count_calls` and expose a read-only count function.
5. Draw the name/object graph before and after `f = d(f)`.

1.16.2 Medium

1. Build `validate_types` using annotations; define behavior for missing annotations.
2. Build a TTL cache with a monotonic clock, maximum size, and explicit clear operation.
3. Implement exponential retry with jitter and a total deadline; test idempotency assumptions.
4. Write a decorator that supports both `@trace` and `@trace(level="debug")`. Explain the ambiguity.
5. Implement a method decorator that maintains independent counters per instance without preventing garbage collection.
6. Write sync and async timing variants with a shared reporting policy.

1.16.3 Hard and interview-grade

1. Preserve a target signature for static typing with `ParamSpec`; discuss runtime introspection separately.
2. Implement a descriptor-aware class-based decorator with `__get__` and per-function state.
3. Build a transaction decorator supporting nested savepoints, rollback, and retry ordering.
4. Design a distributed rate limiter decorator; identify which work cannot safely live in process memory.
5. Decorate generators so time and exceptions cover the full iteration lifecycle, including early close.
6. Build a registry decorator resilient to duplicate names, reloads, and plugin import failures.

1.16.4 Debugging and output prediction

```
1 def mark(label):
2     print("evaluate", label)
3     def decorate(function):
4         print("apply", label)
5         @wraps(function)
```

```

6     def wrapper():
7         print("enter", label)
8         result = function()
9         print("exit", label)
10        return result
11    return wrapper
12    return decorate
13
14    @mark("A")
15    @mark("B")
16    def task():
17        print("task")
18
19    task()

```

Predict before running, then explain why the order is evaluate A, evaluate B, apply B, apply A, enter A, enter B, task, exit B, exit A.

1.16.5 Capstone mini-projects

Build (1) an observable API policy stack with authentication, permission, rate limiting, retry, timing, and redacted structured logging; (2) a scientific-computation cache with stable keys and invalidation; (3) a plugin registry using class decorators. For each, submit a policy-order diagram, failure model, concurrency strategy, tests, and a justification for every decorator rather than explicit code or a context manager.

1.17 Industry Applications

Framework decorators usually perform registration, attach metadata, adapt call signatures, or generate/replace classes. Exact APIs evolve, but the architectural roles are stable.

Ecosystem	Typical role	Why a decorator fits
Django	View permissions, caching, HTTP-method policy, model/admin registration.	Policies sit at request or definition boundaries.
Flask	Route registration on an application object.	URL metadata belongs beside the view definition.
FastAPI	Path-operation registration and dependency/validation metadata.	Signatures and annotations describe the API contract.
pytest	Marks, fixtures, parametrization.	Tests gain collection/execution metadata without custom subclasses.
Click	Commands, options, arguments.	Nested decorators construct a CLI schema near the function.
Typer	Command registration using type annotations.	Function signatures become CLI contracts.
Celery	Task registration and execution options.	A function is promoted into a remotely managed task abstraction.
TensorFlow	Tracing/compilation transformations.	A Python callable becomes a graph-oriented execution unit under constraints.
PyTorch	Gradient-mode controls, compilation/transformation helpers.	Execution policy can surround model functions.

SQLAlchemy	Mapping/events/validation in selected APIs.	Definition-time metadata links Python objects to persistence behavior.
dataclasses	Class transformation generating methods and field behavior.	The whole class definition is the transformation input.
attrs	Class transformation, fields, validators, converters.	Declarative class metadata drives generated behavior.

These libraries rely on decorators because Python executes definitions as ordinary runtime statements. A decorator can observe the newly created function/class, register it, attach metadata, or replace it while keeping configuration adjacent to the definition. The cost is import-time behavior and framework-specific magic, so mature tools invest in introspection, clear errors, and documentation.

1.18 When Decorators Are and Are Not the Right Tool

Use a decorator when behavior applies consistently to a whole callable/class, is orthogonal to its core purpose, composes cleanly, has an explicit order, and remains understandable at the definition site.

Prefer another construct when:

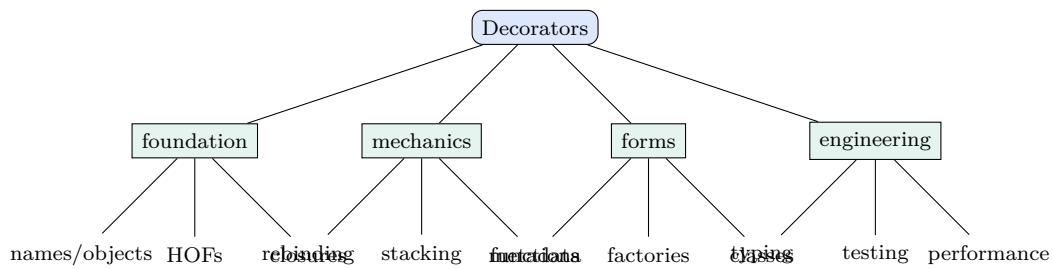
- only part of a function needs resource management—use a context manager;
- the caller should visibly choose behavior per call—use an explicit helper/strategy;
- the behavior defines an object’s enduring responsibilities—use composition or a class;
- control flow is complex or order-dependent enough that nesting becomes obscure—write orchestration explicitly;
- security could be bypassed through undecorated aliases or alternate boundaries—enforce it at the trusted boundary;
- exact signatures, descriptors, generator/coroutine protocols, or debugging transparency cannot be preserved adequately.

1.19 Complete Cheat Sheet and Revision System

1.19.1 Quick reference table

@d	f = d(f) after function creation.
@d(x)	f = d(x)(f); d(x) is evaluated first.
Stack	@a @b def f means f = a(b(f)).
Generic wrapper	Accept/forward *args, **kwargs; return the result.
Metadata	Apply @wraps(target) to the wrapper.
Closure	Wrapper keeps target/configuration alive through cells.
Method	A function wrapper retains descriptor binding behavior.
Class decorator	Receives the newly created class and returns the bound object.
Callable class	Stores state in attributes and implements __call__; methods may need __get__.
Inspection	inspect.signature, inspect.unwrap, __wrapped__, __closure__.
Typing	Callable[P, R] -> Callable[P, R] with ParamSpec/TypeVar.

1.19.2 Mind map



1.19.3 Decision tree

```

Does behavior surround an entire callable/class?
no -> explicit code or context manager
yes -> Is it orthogonal and reusable?
    no -> keep it in the core abstraction
    yes -> Can order, side effects, and protocol be clear?
        no -> explicit composition/orchestration
        yes -> decorator candidate
            configuration? -> decorator factory
            substantial state? -> class-based decorator
            simple capture? -> closure decorator
  
```

1.19.4 CPython internal flow summary

```

source -> compilation creates code objects
module execution -> evaluate decorator expressions
    -> create function object (code + globals + defaults + closure)
    -> call decorators, nearest first
    -> bind public name to final returned object
later call -> wrapper frame(s) -> target frame -> unwind
lifetime -> references/cells keep targets alive; refcount + cyclic GC reclaim
  
```

1.19.5 Revision, design, and interview checklists

I can explain

- names versus objects;
- first-class functions and closures;
- `@d` without calling it magic;
- definition-time versus call-time;
- stacking's three orders;
- descriptors and method binding;
- cells, frames, code/function objects;
- metadata and `__wrapped__`.

Before shipping

- policy has one responsibility;
- arguments/results/exceptions preserved;
- metadata and typing preserved;
- async/generator behavior decided;
- ordering documented and tested;
- mutable state synchronized/scoped;
- secrets excluded from logs/cache keys;
- overhead measured if hot;
- explicit alternative considered.

Final perspective. Decorators are not the at-sign. They are a consequence of Python's object model: definitions create objects, names can be rebound, callables can accept and return callables, and closures preserve context. The at-sign makes a transformation visible where the definition is written. Mastery means more than constructing a wrapper: it means preserving the callable's contract, understanding when transformation occurs, controlling state and ordering, respecting descriptors and async/generator protocols, measuring overhead, debugging the wrapper chain, and choosing explicit code when a decorator would hide more than it clarifies.